

OyaGo MVP Blueprint

Enugu-first keke ride-hailing platform

On-demand booking + passenger fare proposals + driver negotiation



Updated product direction | Python backend + React frontend

Initial operating corridor: Garriki ↔ Holy Ghost ↔ Ogbete, Enugu

Business model: Operate as an Enugu transport business first; defer SaaS expansion.

Prepared 7 November 2026 • Revised to incorporate the supplied brand identity, app showcase and latest product decisions.

1. Executive decision

OyaGo should be built as a **single-market ride-hailing MVP**, not as a SaaS platform. The first objective is to make reliable keke trips happen in a tightly managed Enugu operating area. SaaS, multi-city support and complex tenant management are explicitly deferred until the Enugu business demonstrates repeat demand, driver retention and positive unit economics.

The founder has confirmed responsibility for hands-on testing. AI coding tools may accelerate implementation, but the founder remains responsible for product decisions, field validation, security review, operational readiness and release approval.

The key business risk remains marketplace liquidity: enough eligible drivers must be available where and when passengers request rides. A polished app cannot compensate for a thin driver network.

Operating principle: Build a transport operation supported by software. Do not mistake a completed codebase for a functioning transport business.

Decisions now treated as the working baseline

Decision	Current direction
Market	Enugu, Nigeria; profitable local operation first
Initial corridor	Garriki ↔ Holy Ghost ↔ Ogbete; exact boundaries and hours to be field-defined
Product	Keke ride-hailing with immediate booking and fare negotiation
Business scope	Single operator/business deployment; no SaaS or multi-tenant architecture in MVP
Backend	Python + FastAPI
Frontend	React Native mobile applications; React web admin dashboard
Data and geospatial	PostgreSQL + PostGIS + H3
Payments	Cash paid directly to driver at launch; commission tracked in ledger
Commercial hypothesis	First 30 days at 0% commission per driver, then proposed 5% of completed fares; validate with drivers

2. Brand and supplied design review

The supplied visual direction gives OyaGo a recognisable identity: yellow, black and white; a keke-led mark; bold wordmark; and the tagline **“Your ride, your way.”** The showcase communicates two sides of the marketplace: a passenger trip screen and a driver incoming-request screen.

What is working

- Strong, memorable yellow-and-black palette that can remain consistent across passenger, driver and admin products.
- The keke icon makes the transport category immediately understandable.
- The passenger screen communicates pickup, destination, map, driver identity and trip actions.
- The driver screen makes the incoming request and accept/ignore actions easy to understand.
- The tagline supports the fare-choice proposition without overexplaining the product.

What must be improved before treating the mockup as a build specification

- The passenger flow shown does not demonstrate fare negotiation. Add a clear choice between “Book at suggested fare” and “Offer your fare”, plus driver counteroffers and final fare confirmation.
- The driver request card must show the proposed fare, estimated pickup distance/time, trip destination or destination area, and a short decision timer before acceptance.
- Add a clear trip PIN or dual-confirmation step before a trip starts and a completion confirmation at the end.
- Add driver verification status, vehicle/plate details, passenger support and a safety/report flow.
- The “Easy Payments” claim may imply digital payments. The MVP starts with cash unless a payment provider is deliberately added and tested. Label the payment method clearly.
- The displayed 4-minute ETA and 4.9 rating are illustrative mockup data, not claims OyaGo should publish before real measurements exist.
- Design for low-end Android phones, small screens, poor network conditions, readable text and accessible touch targets.

Design rule: Use the supplied showcase as the visual reference, not as a literal screen-by-screen product specification. The real interface must reflect the actual MVP workflow and verified data.

Recommended UI system

Element	Direction
Primary	OyaGo yellow; reserve for primary actions, highlights and map markers
Base	Near-black / charcoal for driver mode and strong text; white/light grey for passenger surfaces
Typography	Simple, highly readable sans-serif; avoid overly decorative text in operational screens
Buttons	One obvious primary action per screen; secondary actions visually quieter
Status	Consistent labels for requested, offered, accepted, arriving, arrived, in progress, completed and cancelled
Brand assets	Use the supplied transparent logo; prepare separate icon-only, horizontal and app-icon exports

3. Initial service area and launch operations

Updated launch corridor: Garriki ↔ Holy Ghost ↔ Ogbete. This is a corridor-based pilot, not an unrestricted Enugu-wide launch. The exact geofence must be drawn after walking/driving the area, checking practical keke access and confirming driver willingness to serve each segment.

Recommended operating structure

- **Core cluster:** Holy Ghost and Ogbete, where the first reliable driver concentration should be established.
- **Extension:** Garriki–Holy Ghost corridor, added only with drivers who actually work that end of the corridor.
- Define a service polygon and operational hours. Do not show a pickup as serviceable simply because it falls inside a large circle.
- Track supply separately by micro-zone and time window. Ten drivers spread thinly across a wide area may provide worse service than six drivers concentrated in one cluster.
- Keep manual dispatch/support available when no eligible driver is found.

Field decisions required before the geofence is final

Question	What to establish
Driver supply	Where drivers wait, where they travel, and which pickup points they will accept
Route access	Which streets/roads are practically usable by keke; identify restrictions and difficult turns
Demand	Common pickup/drop-off points, peak periods and likely repeat-trip sources
Operating hours	Start with hours when drivers are reliably available; extend only after observing demand
Local acceptance	Speak with relevant park leaders, driver groups and transport stakeholders before public operations
Safety and compliance	Verify applicable permits, insurance, vehicle/driver documents and local operating restrictions

Do not promise “Garriki to anywhere in Enugu” at launch. Advertise only the areas and hours the live operation can serve consistently.

4. MVP product scope

The MVP should solve one complete journey: passenger requests a keke, drivers receive the request, a fare is agreed, the driver picks up the passenger, the trip is completed, and the system records the outcome and commission obligation.

Passenger app

- Phone-number sign-in and basic account profile.
- Pickup and destination selection by map pin, searchable place and draggable marker.
- Fare choice: accept suggested fare or propose an amount.
- View driver acceptance/counteroffer; accept or reject; show final agreed fare before trip confirmation.
- See assigned driver's name/photo, vehicle type and plate details.
- See trip status and location updates when the driver is assigned and sharing location.
- Cash payment label, trip receipt/history, cancellation reason and support/report action.

Driver app

- Phone sign-in, profile, vehicle details and verification status.
- Online/offline toggle with clear location-sharing explanation.
- Incoming request with pickup location, destination area, pickup distance/ETA estimate, proposed fare and timer.
- Accept, reject or counteroffer; prevent multiple active trip assignments.
- Trip navigation handoff, arrived action, trip PIN/dual confirmation, start and complete trip actions.
- Earnings summary, commission ledger, amount due, settlement history and support.

Admin web dashboard

- Approve, reject, suspend and re-verify drivers and vehicles.
- View active drivers, last-seen locations, active trips and service-zone coverage.
- Manually assign/reassign a driver and handle no-driver-found cases.
- Review trip events, cancellations, complaints, disputes and safety incidents.
- View commission ledger, driver balances and settlement records; record cash settlements with audit trail.
- Configure operating hours, service-zone boundaries, fare rules, commission policy and feature flags.

Explicitly defer from MVP

SaaS/multiple transport companies, multi-city operations, iOS, wallets, stored-value balances, card payments, subscriptions, loyalty/rewards, surge pricing, complex route optimisation, in-app chat, microservices and AI-based dynamic pricing. Add features only when real usage proves they are needed.

5. Technical architecture — Python backend + React frontend

Use a **modular monolith**: one Python backend application with clear internal modules, one PostgreSQL database, and one mobile frontend codebase with passenger/driver roles. This is easier to build, test and operate than microservices at MVP stage.

Layer	Recommended technology	Responsibility
Mobile frontend	React Native + TypeScript	Passenger and driver experiences; Android-first
Admin frontend	React + TypeScript	Operations dashboard for desktop/tablet
API/backend	Python + FastAPI	Authentication, permissions, booking, negotiation, matching, trip lifecycle, ledger
Validation	Pydantic	Request/response schemas and data validation
Database access	SQLAlchemy 2.x + Alembic	ORM/data access and controlled schema migrations
Database	PostgreSQL	System of record for users, drivers, trips, fares and money
Geospatial	PostGIS + Python H3 library (h3-py)	Exact spatial filters, service polygons, H3 cell indexing
Realtime	FastAPI WebSockets initially	Driver location and trip/offer updates; add managed realtime only if it materially reduces work
Background jobs	Simple task runner initially; Redis/Celery later if needed	Expiry, reminders, settlement tasks and asynchronous work
Maps	Google Maps SDK / Routes API behind an adapter	Map display, geocoding and shortlisted route-time estimates
File storage	Private object storage	Driver/vehicle verification evidence with strict access
Deployment	Linux VPS or managed hosting, HTTPS, backups, monitoring	Production runtime and operational recovery

Important architecture rules

- Python backend is authoritative. Never trust the mobile app to decide fare totals, trip completion, driver assignment or commission.
- Keep secrets and provider API keys on the server; restrict mobile keys by package/signature and API where supported.
- Use migrations for database changes. Never let an AI coding agent freely alter production data or schema.
- Keep business logic separate from HTTP routes: authentication, driver, location, booking, offers, matching, trips, commission, settlements, admin and audit modules.
- Use one codebase/repository with clear folders and automated tests. Avoid premature microservices.

6. H3 + PostGIS driver matching

H3 is a spatial indexing aid, not the matching engine by itself. It groups nearby driver locations into cells so the backend can find a small candidate set efficiently. PostGIS then applies exact geographic and service-area checks. A routing provider is called only for the shortlisted candidates.

Recommended request-to-driver sequence

Step	Backend action
1. Location intake	Accept driver GPS updates only when the driver is online/available; store coordinates, accuracy and timestamp.
2. H3 indexing	Compute the driver's H3 cell using the selected resolution; store it with the location record.
3. Candidate discovery	Search the pickup cell and neighboring cells; expand progressively if too few candidates exist.
4. Exact filtering	Use PostGIS distance and service-zone checks; reject stale, inaccurate, offline, busy, unverified, suspended or ineligible drivers.
5. Shortlist	Keep a small candidate set, initially around 5–10; tune with real driver density and measured latency.
6. ETA/ranking	Request road ETA for only shortlisted drivers; rank using ETA plus acceptance/reliability signals, not ETA alone.
7. Atomic assignment	Reserve the selected driver transactionally and re-check availability to prevent double booking.
8. Fallback	Progressively expand within allowed service boundaries or alert admin for manual dispatch.

H3 resolution and search radius must be tuned using actual Enugu supply density. Resolution 9 is a starting experiment, not a fixed design truth. H3 cell boundaries do not equal a circle or a road travel radius.

Routing cautions

- Google Compute Route Matrix can compare several driver origins against one pickup, but cost is based on route elements; enforce quotas, budgets and small candidate lists.
- Do not assume car or two-wheel routing accurately models three-wheel keke access. Field-test real pickup routes and compare predicted ETA with actual arrival time.
- Keep routing behind a provider interface. If results are poor, show a conservative estimate or no precise ETA rather than false precision.
- GPS updates should be rate-limited and validated for freshness, accuracy and impossible jumps. Do not write every tiny movement as an expensive database operation.

Non-negotiable: assignment must be atomic. Two simultaneous booking requests must never successfully reserve the same driver.

7. Data model, trip lifecycle and fare negotiation

Core database entities

Entity	Purpose
users	Phone identity, role, account status and timestamps
driver_profiles	Driver details, verification state, rating summary and availability
vehicles	Vehicle type, plate, photos and verification status
driver_locations	Coordinates, H3 cell, accuracy, recorded_at and last-seen status
service_zones	Named pilot zones, polygon boundary, operating hours and active flag
ride_requests	Passenger, pickup, destination, proposed/suggested fare, status and expiry
ride_offers	Driver, offered amount, counteroffer, status and expiry
trips	Assigned passenger/driver, agreed fare, trip status and timestamps
trip_events	Append-only event history for important trip actions
commission_ledger	Completed trip, commission rate, amount, status and settlement linkage
settlements	Driver, amount, payment reference, settlement date and admin actor
disputes / incidents	Reported issue, category, evidence, status and resolution
audit_logs	Sensitive admin and financial actions

Trip lifecycle

requested → **matching** → **offered** → **accepted** → **driver_arriving** → **arrived** → **in_progress** → **completed**
Terminal outcomes: cancelled, expired, no_driver_found, disputed/refunded.

Enforce legal state transitions on the backend. Expire offers; prevent simultaneous active assignments; require the agreed fare to be confirmed before trip start; make completion idempotent so retries cannot create duplicate commission entries.

Fare negotiation rules

- Suggested fare is clearly labelled as an estimate, not a guaranteed price.
- Passenger can submit an offer; driver may accept, reject or counter within defined limits.
- Passenger must explicitly accept a counteroffer before assignment is final.
- Store every offer/counteroffer with actor, amount, timestamp and expiry. Never overwrite negotiation history.
- Once confirmed, the agreed fare is locked for that trip unless both parties confirm a permitted change.
- Do not introduce surge pricing or opaque fare changes in the MVP.

Store all money as integer kobo, never floating-point naira. Use database constraints, foreign keys, indexes and immutable ledger entries. A driver balance is an accounting record; it is not cash received by OyaGo.

8. Revenue, cash collection and safety

Working commercial hypothesis

0% commission for each driver's first 30 calendar days, then a proposed 5% commission on completed ride fares. No commission on cancelled, failed or refunded rides. Communicate the policy before onboarding and test whether drivers consider it fair.

Ride fare	5% commission
■500	■25
■1,000	■50
■2,000	■100
■3,000	■150

These are gross amounts before hosting, maps, data, support, field operations, refunds, taxes and unpaid balances. Five percent is a hypothesis, not proven optimal.

Cash commission weakness

- Passenger pays the driver directly in cash; OyaGo does not automatically receive its commission.
- Create a ledger entry only after backend-confirmed trip completion.
- Show drivers the fare, commission, running balance and settlement history.
- Set a low unpaid-balance cap, clear settlement frequency, reminders and a documented dispute process.
- Record every cash settlement with date, amount, receiving staff member and driver confirmation.
- Do not introduce a wallet or hold customer funds until payment-provider and regulatory implications have been reviewed.

Safety, fraud and privacy

- Verify driver identity and vehicle information in person before activation; re-check when needed.
- Use trip PIN or dual confirmation to start and complete trips.
- Restrict access to passenger/driver phone numbers and identity documents; use role-based admin permissions.
- Record key trip events and admin actions. Provide a clear report/help route.
- Define how long precise location and verification files are retained, who can access them and how deletion requests are handled.
- Create a manual incident escalation process before public launch. Do not claim a safety feature exists until it is implemented and tested.

You cannot completely prevent a driver and passenger from arranging future rides off-platform. Reduce the incentive through reliable bookings, support, trip history, trusted-driver rebooking and fair driver economics; do not promise perfect anti-bypass protection.

9. Build and launch roadmap

AI coding agents can accelerate implementation, but delivery time depends on integration, bugs, mobile permissions, mapping, deployment and test results. Treat the milestones below as acceptance gates, not guaranteed calendar promises.

Milestone	Build outcome	Exit criteria
0. Product decisions	Geofence, operating hours, driver verification, fare rules, commission and support process	Founder signs off written decisions; local operating checks underway
1. Foundation	Repo, environments, auth, roles, database, migrations, logging and admin skeleton	Automated tests pass; staging deployment works
2. Driver supply	Driver onboarding, verification, online/offline, GPS intake and location freshness	Test devices update reliably; stale/fake locations rejected
3. Booking + negotiation	Passenger request, driver offers/counteroffers, expiry and agreed fare	Full booking flow passes concurrent and edge-case tests
4. Matching + trip	H3 candidate search, PostGIS filters, routing shortlist, atomic assignment and trip states	No double assignment; route/ETA behaviour tested on actual keke routes
5. Finance + operations	Commission ledger, settlement records, disputes, admin actions and audit trail	Ledger reconciles; cancelled/refunded trips do not create commission
6. Controlled pilot	Limited drivers/passengers in the defined corridor, manual support and monitoring	Reliable service and acceptable operational metrics demonstrated
7. Improve and expand	Fix observed failures; extend corridor or hours only when supply supports it	Existing zone remains stable for multiple weeks

Recommended MVP development order

Build the backend foundation and database first, then driver location and availability, then booking/negotiation, then matching and trip lifecycle, then commission/admin operations. Build the polished UI alongside these flows, but do not let visual work hide incomplete backend rules.

10. Testing plan and success metrics

The founder will own hands-on testing. Testing should be continuous, not postponed until the end. Use local development, staging and production environments; never test destructive migrations against production data.

Minimum test categories

- Unit tests: fare calculations, commission calculations, state transitions, offer expiry and eligibility rules.
- API tests: authentication, permissions, invalid payloads, duplicate requests and error handling.
- Concurrency tests: two passengers requesting the same driver; repeated accept taps; retries after network loss.
- Geospatial tests: H3 neighboring-cell search, PostGIS radius checks, zone boundaries and GPS accuracy/freshness.
- Mobile tests: background location, app minimisation, low battery, permission denial, low-end Android devices and poor connectivity.
- Operational tests: manual dispatch, driver suspension, cancellation, refund/dispute, settlement reconciliation and incident reporting.
- Field tests: actual routes between Garriki, Holy Ghost and Ogbete; compare route estimates with real pickup times.

Pilot targets — hypotheses to test, not promises

Metric	Initial test target / signal
Verified active drivers	10–15 in the launch corridor; count drivers who actually go online, not registrations
Valid requests matched	Aim for 70%+ once pilot operations stabilise
Matched trips completed	Aim for 60%+; investigate every major failure reason
Pickup reliability	Measure median and 90th percentile actual pickup time
Repeat passengers	Evidence of repeat bookings within 30 days
Driver retention	Drivers continue using the service after free period
Commission collection	Track collected amount versus ledger amount due
Safety/support	Track complaints and incidents per 100 completed trips; resolve every serious incident

Do not scale based on downloads, registrations or social-media attention. Scale based on completed rides, repeat use, driver retention, pickup reliability and collected revenue.

11. Main risks, loopholes and controls

Risk / loophole	Control
Too few drivers at request time	Concentrate geography and hours; recruit and verify before marketing
Recruiter becomes a single point of failure	Founder builds direct relationships; recruit through multiple parks, referrals and local partners
Driver signs up but stays offline	Track active online hours and completed rides, not registration totals
Cash commission is not paid	Ledger, clear settlement schedule, balance cap and fair suspension/dispute policy
Driver/passenger bypasses platform	Deliver repeat value and support; accept that perfect prevention is impossible
Stale or spoofed GPS	Timestamp, accuracy and plausibility checks; server-side eligibility rules
Double-booked driver	Transactional reservation, database constraints and active-trip guard
Wrong ETA for keke	Field-test actual routes; conservative fallback; provider abstraction
API bill grows unexpectedly	Shortlists, quotas, budgets, caching where appropriate and usage alerts
Poor network	Retry-safe APIs, clear stale state, lightweight screens and manual support fallback
Security/privacy incident	Least privilege, secure storage, audit logs, backups and incident response
Local legal/operating issue	Verify permits, route restrictions, insurance and transport requirements with relevant local authorities/professionals

Regulatory note

The earlier source blueprint did not establish authoritative current Enugu transport requirements. This document therefore treats permits, park/route restrictions, insurance, driver/vehicle documents and data-protection obligations as checks to complete—not as legal conclusions. Obtain local professional guidance before public operation.

12. Funding priorities and SaaS boundary

If raising or sourcing money for development, ask for enough to cover a defined MVP build-and-pilot budget, not an open-ended technology project. Separate development costs from operating costs.

Priority	Spend on
1. Development tools	One primary coding agent subscription; GitHub/source control; essential developer tooling
2. Core infrastructure	Database/hosting, domain, backups, logs and monitoring
3. Maps and communications	Small controlled API budget, SMS/OTP if used, data for field testing
4. Field operations	Driver recruitment, transport allowance, verification and local coordination
5. Testing and launch	Test devices if needed, bug fixing, support materials and controlled pilot

Do not fund

- Expensive branding campaigns before service reliability is demonstrated.
- Multiple AI coding subscriptions before one tool is being used effectively.
- Microservices, Kubernetes or complex infrastructure for a small MVP.
- Multi-city or multi-tenant SaaS features before there are multiple paying transport operators.
- Digital wallet/payment complexity before the cash-ledger workflow is understood.

When SaaS becomes a sensible next step

Only consider SaaS after OyaGo has a stable Enugu operation, documented onboarding/dispatch/settlement processes, reliable unit economics and evidence that other transport operators want to pay for the same system. At that point, assess tenant isolation, operator-specific configuration, billing, support boundaries, data ownership and compliance. Do not build these abstractions prematurely.

The immediate product is OyaGo's own operating platform. A future SaaS product is a separate business decision that must be earned by evidence.

13. Instructions for Claude Code / Codex

Use this document as the product and architecture baseline. Do not ask the coding agent to generate the entire system in one giant pass. Work milestone by milestone and require evidence that each milestone works.

- Before coding, inspect the repository and propose a folder structure, data model and implementation plan. Wait for approval before large changes.
- Use Python + FastAPI for backend business logic; React Native + TypeScript for Android-first passenger/driver apps; React + TypeScript for the admin web dashboard.
- Implement a modular monolith. Do not introduce microservices unless a measured requirement justifies them.
- Write migrations, tests and concise documentation with every feature.
- Keep all business-critical rules server-side: fare negotiation, driver eligibility, matching, assignment, trip transitions, commission and settlement.
- Use H3 for candidate discovery and PostGIS for exact distance/zone filtering. Make H3 resolution and search expansion configurable.
- Put routing behind an adapter and keep provider calls limited to shortlisted candidates.
- Use transactions/locking/constraints to prevent double booking and duplicate commission entries.
- Never hardcode API secrets, credentials, production URLs or test personal data in the repository.
- Never run destructive production commands, migrations or data edits without explicit human approval.
- At the end of each milestone, report changed files, migrations, tests run, test results, known gaps and manual steps required.

Suggested repository layout

```
backend/
app/api/ # FastAPI routers
app/core/ # config, auth, permissions
app/models/ # SQLAlchemy models
app/schemas/ # Pydantic schemas
app/services/ # matching, trips, fares, ledger
app/repositories/ # database access
app/tasks/ # expiry and background work
migrations/ # Alembic
tests/
mobile/ # React Native passenger/driver app
admin/ # React web dashboard
docs/ # product, API and operations docs
```

Definition of done

A feature is not done because the screen renders. It is done when the backend rule exists, permissions are enforced, edge cases are handled, tests pass, errors are observable, and the relevant flow works on a real Android device or staging environment.

14. Decisions still requiring founder sign-off

Decision	Must be settled before live launch
Exact service polygon	Draw and test Garriki–Holy Ghost–Ogbete boundaries; decide what is out of area
Operating hours	Choose initial hours based on driver supply and local operating restrictions
Driver recruitment lead	Choose a trusted coordinator, payment terms and performance-based targets; retain direct driver relationships
Driver verification	List required identity/vehicle evidence and who checks it
Fare rules	Set suggested fare source, minimum/maximum bounds, counteroffer limits and expiry times
Cancellation rules	Define passenger/driver cancellation reasons, timing and any penalties
Commission settlement	Set payment frequency, unpaid balance cap, evidence and dispute process
Support and safety	Set support contact, incident escalation and emergency response process
Legal/insurance	Confirm relevant Enugu operating requirements with qualified local sources
Maps budget	Set daily/monthly API limits and alerts before field use

Final recommendation

Proceed with the OyaGo MVP using Python for the backend and React for the frontend, with React Native for Android passenger/driver experiences and a React web admin dashboard. Keep the product single-market and operationally focused. Include Garriki as a measured extension of the Holy Ghost–Ogbete core corridor, not as a promise of unrestricted city coverage.

The supplied brand direction is a good starting point. The next work is to turn it into complete, testable passenger, driver and admin flows, then build the backend rules that make those flows safe and reliable.

Build the smallest complete ride operation—not the largest possible app.

Technical references retained from the original blueprint

Google Routes API usage and billing: https://developers.google.com/maps/documentation/routes/usage-and-billing
Google Compute Route Matrix: https://developers.google.com/maps/documentation/routes/compute_route_matrix
Uber H3 documentation: https://github.com/uber/h3
H3 resolution reference: https://github.com/uber/h3/blob/master/website/docs/library/restable.md
Supabase billing documentation (if Supabase is selected for managed database/realtime):
https://supabase.com/docs/guides/platform/billing-on-supabase

This revised blueprint incorporates the earlier OyaGo business/technical blueprint, the founder's latest decisions and the two supplied visual assets. It is a planning document, not legal advice or a guarantee of commercial success.